

ThinkAI 视频生成 API 接口文档

本文档覆盖 ThinkAI 视频生成接口支持的 CH1、CH3、CH9 视频模型。

1. 接入信息

项目	值
Base URL	https://www.thinkai.tv
鉴权方式	Bearer Token
请求格式	application/json

所有请求都需要在请求头中携带 ThinkAI API Key：

Authorization: Bearer YOUR_THINKAI_API_KEY

创建任务时还应携带：

Content-Type: application/json

API Key 请妥善保管，不要写入公开仓库、客户端安装包或其他可被公开访问的位置。

2. 支持模型

当前支持 9 个模型。模型 ID 与 resolution 必须严格对应，不能交叉组合。

分组	模型 ID	分辨率	纯文本	图片/视频/音频参考
CH1	ch1-sd-2.0-720p	720p	支持	最多 9 / 3 / 3
CH1	ch1-sd-2.0-1080p	1080p	支持	最多 9 / 3 / 3
CH1	ch1-sd-2.0-4k	4k	支持	最多 9 / 3 / 3
CH1	ch0102-sd-2.0-1080p	1080p	支持	最多 9 / 3 / 3，总数最多 12
CH3	ch3-sd-2.0-xh	720p	支持	最多 9 / 3 / 3
CH3	ch3-sd-2.0-xh-1080p	1080p	支持	最多 9 / 3 / 3
CH3	ch3-sd-2.0-xh-4k	4k	支持	最多 9 / 3 / 3
CH9	ch9-sd-2.0-ck2-720p	720p	支持	最多 9 / 3 / 3，总数最多 12
CH9	ch0904-sd-2.0-720p	720p	支持	最多 9 / 3 / 3，总数最多 12

2.1 分组差异

约束	CH1	CH3 XH	CH9 CK2
提示词	1-5000 字符	1-4000 字符	10-5000 字符
画幅	16:9、9:16、1:1、3:4、4:3、21:9	16:9、9:16	16:9、9:16
时长	4-15 秒	4-15 秒	10-15 秒
素材总数	最多 15 个	最多 15 个	最多 12 个
纯文本生视频	支持	支持	支持

模型级差异：ch0102-sd-2.0-1080p 素材总数最多 12 个；ch0904-sd-2.0-720p 时长为 5-15 秒，画幅为 16:9、9:16。

3. 接口总览

方法	ThinkAI URL	说明
POST	https://www.thinkai.tv/v1/videos	创建视频生成任务
GET	https://www.thinkai.tv/v1/videos/{task_id}	查询任务状态和进度
GET	https://www.thinkai.tv/v1/videos/{task_id}/content	下载生成的视频

4. 创建视频任务

4.1 请求

POST https://www.thinkai.tv/v1/videos
Authorization: Bearer YOUR_THINKAI_API_KEY
Content-Type: application/json

4.2 请求参数

参数	类型	必填	说明
model	string	是	支持的模型 ID，见第 2 节
prompt	string	是	长度限制按模型分组区分
mode	string	是	固定传 references，不支持 frames

参数	类型	必填	说明
client_task_id	string	是	幂等任务 ID，最长 128 字符，只允许字母、数字、下划线、短横线和点
aspect_ratio	string	否	支持范围按模型分组区分，默认 16:9
duration	number	否	支持范围按模型分组区分
resolution	string	否	必须与所选模型固定分辨率一致
references	object	否	参考素材；当前 9 个模型均可不传，表示纯文本生视频

建议调用方显式传入稳定的业务唯一 ID，便于处理重试和任务追踪。

4.3 参考素材

参考素材必须是公网可访问的 HTTPS URL。接口不接收 multipart/form-data 文件、Base64 或 Data URL；本地文件需要先上传到对象存储，再将 URL 放入 references。

素材	单个素材字段	多个素材字段	单类上限
图片	references.image	references.images	9 张
视频	references.video	references.videos	3 条
音频	references.audio	references.audios	3 条

同一类素材只使用单数或复数字段中的一种。只有 1 个素材时使用单数字段，2 个及以上时使用复数字段；不要传空数组或只有 1 个元素的复数字段。

CH9 带参考视频时，参考视频真实时长与生成时长之和最多为 25 秒。

4.4 CH1 创建示例

```
curl -X POST 'https://www.thinkai.tv/v1/videos' \
-H 'Authorization: Bearer YOUR_THINKAI_API_KEY' \
-H 'Content-Type: application/json' \
-d '{
  "model": "ch1-sd-2.0-1080p",
  "prompt": "保持人物外貌一致，参考动作和音乐自然走动，画面稳定",
  "mode": "references",
  "client_task_id": "order_20260721_ch1_000001",
  "references": {
    "images": [
      "https://example.com/person-front.jpg",
      "https://example.com/person-side.jpg"
    ],
    "video": "https://example.com/motion-reference.mp4",
    "audio": "https://example.com/music.mp3"
  }
},
```

```
"aspect_ratio": "9:16",  
"duration": 8,  
"resolution": "1080p"  
}
```

4.5 CH3 创建示例

```
curl -X POST 'https://www.thinkai.tv/v1/videos' \  
-H 'Authorization: Bearer YOUR_THINKAI_API_KEY' \  
-H 'Content-Type: application/json' \  
-d '{  
  "model": "ch3-sd-2.0-xh-4k",  
  "prompt": "人物自然向前走，镜头缓慢推进，保持主体和背景风格一致",  
  "mode": "references",  
  "client_task_id": "order_20260721_ch3_000001",  
  "references": {  
    "image": "https://example.com/reference.jpg"  
  },  
  "aspect_ratio": "16:9",  
  "duration": 10,  
  "resolution": "4k"  
}'
```

4.6 CH9 纯文本示例

```
curl -X POST 'https://www.thinkai.tv/v1/videos' \  
-H 'Authorization: Bearer YOUR_THINKAI_API_KEY' \  
-H 'Content-Type: application/json' \  
-d '{  
  "model": "ch9-sd-2.0-ck2-720p",  
  "prompt": "一位年轻人在城市街道自然行走，镜头稳定推进，光影真实自然",  
  "mode": "references",  
  "client_task_id": "order_20260721_ch9_000001",  
  "aspect_ratio": "16:9",  
  "duration": 10,  
  "resolution": "720p"  
}'
```

4.7 创建响应

```
{  
  "id": "task_abc123def456",  
  "object": "video",  
  "model": "ch1-sd-2.0-1080p",  
  "status": "queued",  
}
```

```
"progress": 0,  
"created_at": 1784592000  
}
```

调用方必须保存响应中的 id，后续使用该值查询任务和下载视频。

5. 查询视频任务

5.1 请求

```
GET https://www.thinkai.tv/v1/videos/{task_id}  
Authorization: Bearer YOUR_THINKAI_API_KEY
```

```
curl 'https://www.thinkai.tv/v1/videos/task_abc123def456' \  
-H 'Authorization: Bearer YOUR_THINKAI_API_KEY'
```

建议每 5 秒查询一次，直到任务进入 completed 或 failed 状态。避免上一轮请求尚未结束时再次发起同一任务的查询。

5.2 完成响应

```
{  
  "id": "task_abc123def456",  
  "object": "video",  
  "model": "ch1-sd-2.0-1080p",  
  "status": "completed",  
  "progress": 100,  
  "created_at": 1784592000,  
  "completed_at": 1784592100,  
  "metadata": {  
    "url": "/v1/videos/task_abc123def456/content"  
  }  
}
```

5.3 失败处理

当 status 为 failed 时，优先读取 metadata.fail_reason。任务失败后停止轮询，并记录 task_id、client_task_id 和失败原因。

6. 下载视频

只有任务状态为 completed 时才应下载视频。

```
GET https://www.thinkai.tv/v1/videos/{task_id}/content
Authorization: Bearer YOUR_THINKAI_API_KEY
```

```
curl -L 'https://www.thinkai.tv/v1/videos/task_abc123def456/content' \
-H 'Authorization: Bearer YOUR_THINKAI_API_KEY' \
--output output.mp4
```

下载接口返回视频二进制流，不是 JSON。可按需携带 Range 请求头进行断点续传或媒体分段读取。

7. 幂等与重试

1. 同一笔业务任务的首次请求和所有重试必须使用相同的 client_task_id。
2. 不要在每次重试时随机生成新值，否则可能重复创建并计费。
3. 不同业务任务必须使用不同的 client_task_id。
4. 重试创建请求时，应使用完全相同的参数和同一个 client_task_id。
5. 不要在重试时修改模型、提示词、素材或其他生成参数。

8. Node.js 完整调用示例

```
const BASE_URL = "https://www.thinkai.tv";
const API_KEY = process.env.THINKAI_API_KEY;
const clientTaskId = "order_20260721_000001";

const headers = {
  Authorization: `Bearer ${API_KEY}`,
  "Content-Type": "application/json"
};

async function createTask() {
  const response = await fetch(`${BASE_URL}/v1/videos`, {
    method: "POST",
    headers,
    body: JSON.stringify({
      model: "ch1-sd-2.0-720p",
      prompt: "让人物自然走动，镜头缓慢推进，电影感",
      mode: "references",
      client_task_id: clientTaskId,
      aspect_ratio: "16:9",
      duration: 5,
      resolution: "720p"
    })
  });
};
```

```

    if (!response.ok) throw new Error(await response.text());
    return response.json();
}

async function getTask(taskId) {
    const response = await fetch(`${BASE_URL}/v1/videos/${taskId}`, {
        headers
    });

    if (!response.ok) throw new Error(await response.text());
    return response.json();
}

const created = await createTask();
let task;

do {
    await new Promise((resolve) => setTimeout(resolve, 5000));
    task = await getTask(created.id);
    console.log(task.status, task.progress);
} while (!["completed", "failed"].includes(task.status));

if (task.status === "failed") {
    throw new Error(task.metadata?.fail_reason || "视频生成失败");
}

const videoResponse = await fetch(
    `${BASE_URL}/v1/videos/${created.id}/content`,
    { headers: { Authorization: `Bearer ${API_KEY}` } }
);

if (!videoResponse.ok) throw new Error(await videoResponse.text());
const video = Buffer.from(await videoResponse.arrayBuffer());
await import("node:fs/promises").then((fs) => fs.writeFile("output.mp4", video));

```